UNIVERSIDAD TÉCNICA DE MANABÍ

FACULTA DE LA EDUCACIÓN DE LAS CIENCIAS INFORMATICAS

CARRERA:

TECNOLOGIAS DE LA INFORMACIÓN

TEMA:

ESTUDIO DE CASO-SISTEMA DE CONTROL DE RESCATE Y ADOPCIÓN ANIMAL (INTEGRANDO ESTANDARES DE MODELADO UML)

DOCENTE:

ING. JORGE ZAMBRANO

AUTOR:

JOHN DAVID PARRALES BARAHONA

NIVEL:

QUINTO

PARALELO:

A

MATERIA:

DESARROLLO DE SISTEMAS INFORMATICOS

Actividad #1: Estudio de Caso – Sistema de Control de Rescate y Adopción Animal

1. Análisis del escenario propuesto

1.1. Descripción del problema

Actualmente, los refugios de animales necesitan llevar un control organizado de los rescates, tratamientos veterinarios y procesos de adopción. Cuando esta información se maneja manualmente, pueden presentarse problemas como pérdida de registros, duplicación de información y dificultades para conocer el estado de cada animal.

Por esta razón, se propone diseñar un Sistema de Control de Rescate y Adopción Animal que permita registrar animales rescatados, asignar responsables, controlar los tratamientos veterinarios y gestionar las solicitudes de adopción.

El sistema estará diseñado para trabajar con diferentes tipos de animales, como perros, gatos y animales silvestres, considerando que estos últimos pueden necesitar procedimientos especiales y derivación a entidades autorizadas.

1.2. Objetivo del sistema

Diseñar la arquitectura lógica de un sistema informático que facilite la administración de rescates y adopciones de animales, utilizando diagramas UML y patrones de diseño que permitan obtener una estructura organizada, escalable y fácil de mantener.

1.3. Identificación de actores

Actor	Función dentro del sistema

Administrador	Gestiona usuarios, supervisa rescates y administra adopciones.
Voluntario	Registra animales rescatados y realiza seguimiento.
Veterinario	Evalúa animales y registra diagnósticos y tratamientos.
Adoptante	Consulta animales disponibles y presenta solicitudes de adopción.

1.4. Requerimientos funcionales

Los requerimientos funcionales describen las operaciones que deberá realizar el sistema.

Código	Requerimiento
RF01	Permitir el registro e inicio de sesión de usuarios.
RF02	Registrar animales rescatados con sus datos principales.
RF03	Registrar la fecha, ubicación y descripción del rescate.
RF04	Asignar voluntarios o veterinarios responsables.
RF05	Registrar diagnósticos y tratamientos veterinarios.
RF06	Actualizar el estado de cada animal.
RF07	Consultar animales disponibles para adopción.
RF08	Registrar solicitudes de adopción.
RF09	Permitir al administrador aprobar o rechazar solicitudes.
RF10	Generar notificaciones cuando cambie el estado de un animal.

1.5. Requerimientos no funcionales

Código	Requerimiento
RNF01	El sistema deberá ser fácil de utilizar.
RNF02	La información deberá estar protegida mediante autenticación y permisos.
RNF03	La arquitectura deberá permitir agregar nuevas funcionalidades.
RNF04	El sistema deberá evitar registros duplicados y mantener la integridad de los datos.
RNF05	El código deberá organizarse en módulos reutilizables.
RNF06	El sistema deberá mantener un historial de los cambios realizados.

2. Modelado del sistema mediante diagramas UML

Los diagramas UML permiten representar visualmente el funcionamiento y la estructura del sistema antes de su desarrollo.

2.1. Diagrama de casos de uso

Este diagrama identifica las acciones que puede realizar cada actor y su interacción con el sistema.

Se establecen las siguientes relaciones:

- El administrador gestiona usuarios, asigna responsables y evalúa solicitudes.

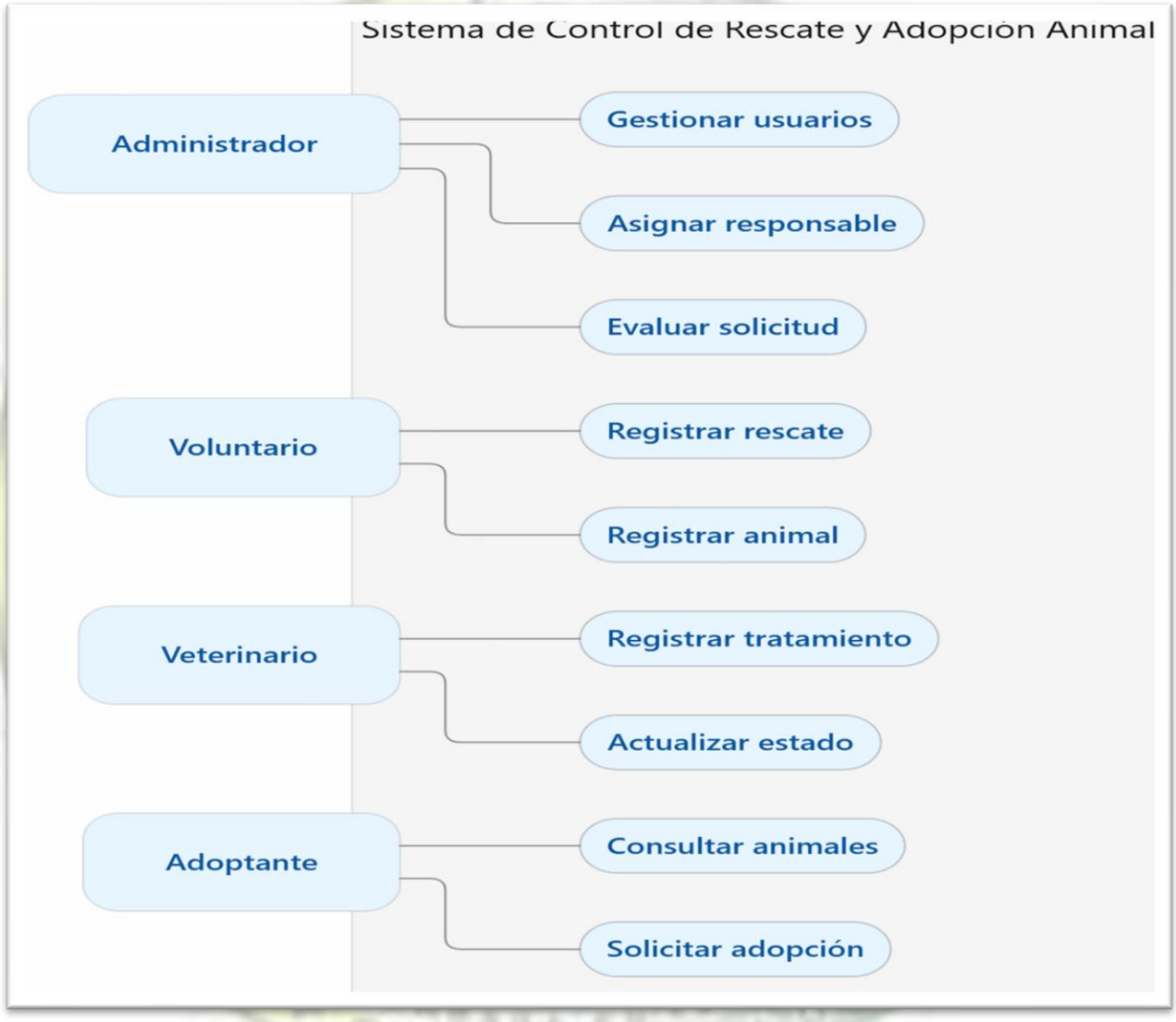
- El voluntario registra animales y rescates.

- El veterinario registra diagnósticos y tratamientos.

- El adoptante consulta animales y solicita adopciones.

Figura 1. Diagrama de casos de uso

Las conexiones indican las funcionalidades correspondientes a cada actor.



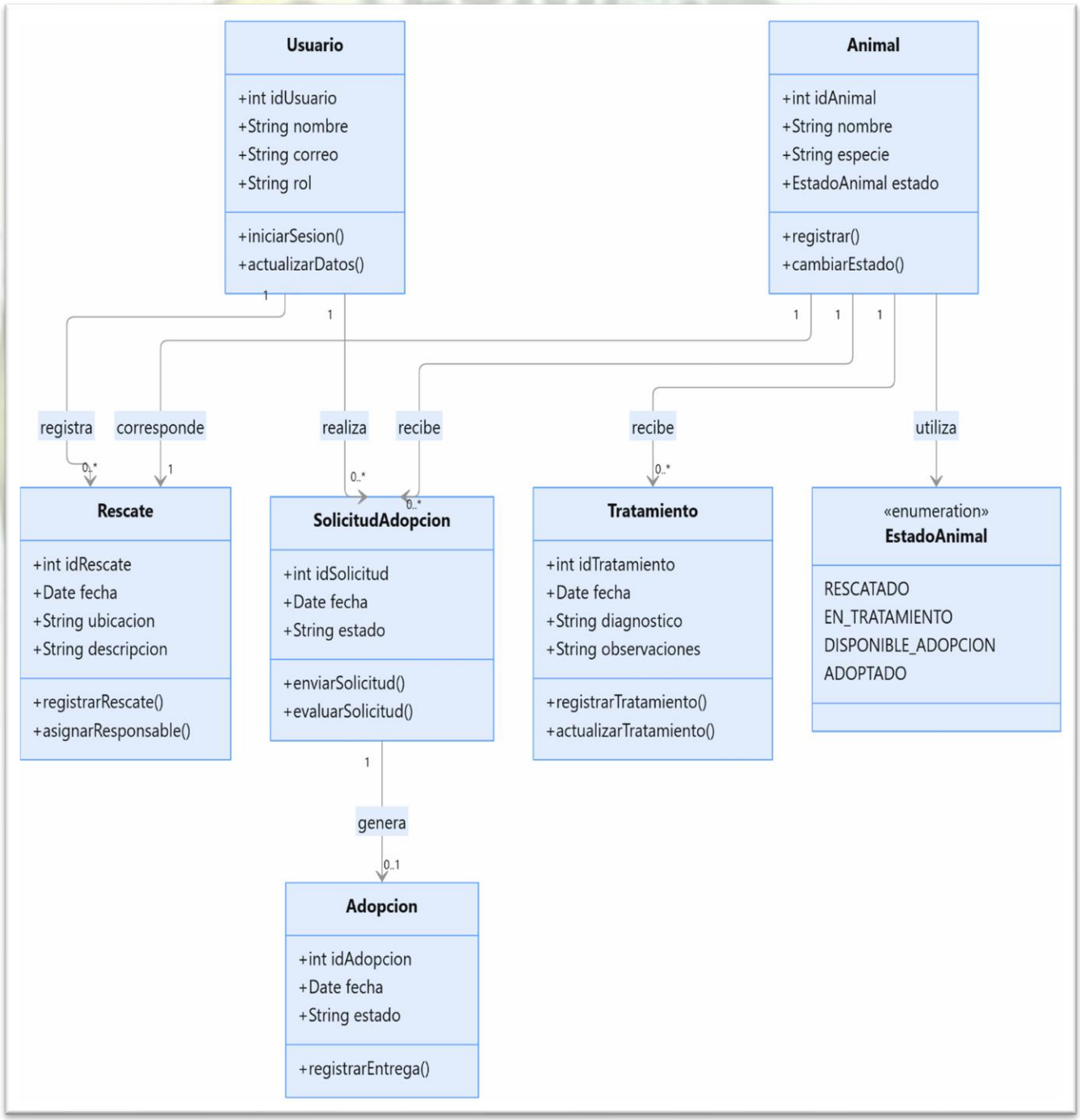
Representación conceptual de casos de uso mediante actores, límite del sistema y funcionalidades

2.2. Diagrama de clases UML

El diagrama de clases representa la estructura interna del sistema, identificando sus clases, atributos, métodos y relaciones.

Las clases principales son Usuario, Animal, Rescate, Tratamiento, Solicitud y Adopción

Figura 2. Diagrama de clases UML



Explicación: La clase Animal es uno de los elementos centrales, ya que contiene la información del animal rescatado y permite conocer su estado actual. Las demás clases se relacionan con ella para gestionar el rescate, la atención veterinaria y la adopción.

La clase Usuario permite representar los diferentes roles del sistema mediante su atributo rol, mientras que Estado Animal define los estados posibles durante el proceso.

3. Aplicación de tres patrones de diseño

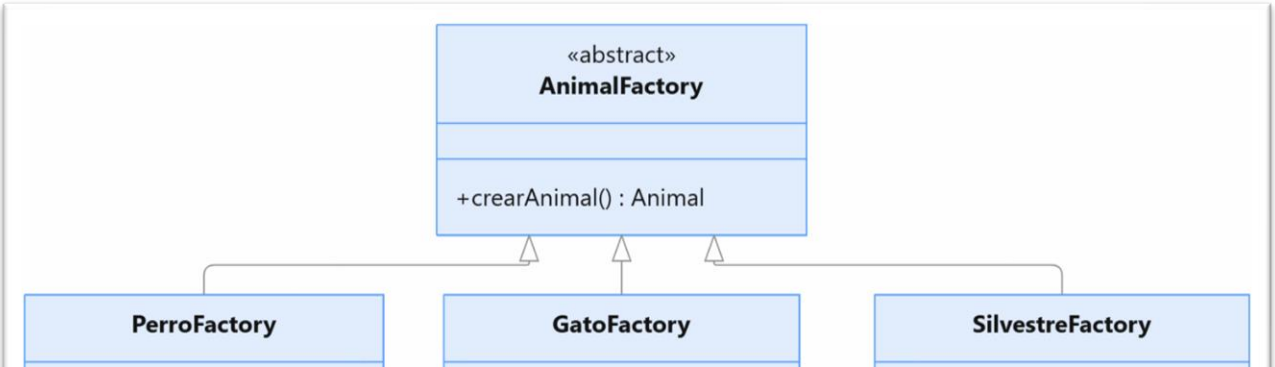
Para mejorar la organización del sistema se seleccionaron tres patrones de diseño pertenecientes a diferentes categorías.

3.1. Patrón creacional: Factory Method

Propósito: Facilitar la creación de diferentes tipos de animales sin depender directamente de sus clases concretas.

En el sistema, cuando un voluntario registra un rescate, se debe crear el objeto correspondiente según la especie del animal.

Se propone utilizar una fábrica abstracta y fábricas concretas para crear perros, gatos y animales silvestres.



Ejemplo: Si el voluntario selecciona la categoría Perro, el sistema utiliza PerroFactory para crear el registro correspondiente.

Beneficio: Facilita incorporar nuevas especies sin modificar la lógica principal del registro.

3.2. Patrón estructural: Facade (Fachada)

Propósito: Simplificar las operaciones complejas del proceso de rescate mediante una interfaz centralizada.

Se propone crear la clase RescateFacade, encargada de coordinar los módulos que intervienen en el registro de un rescate.



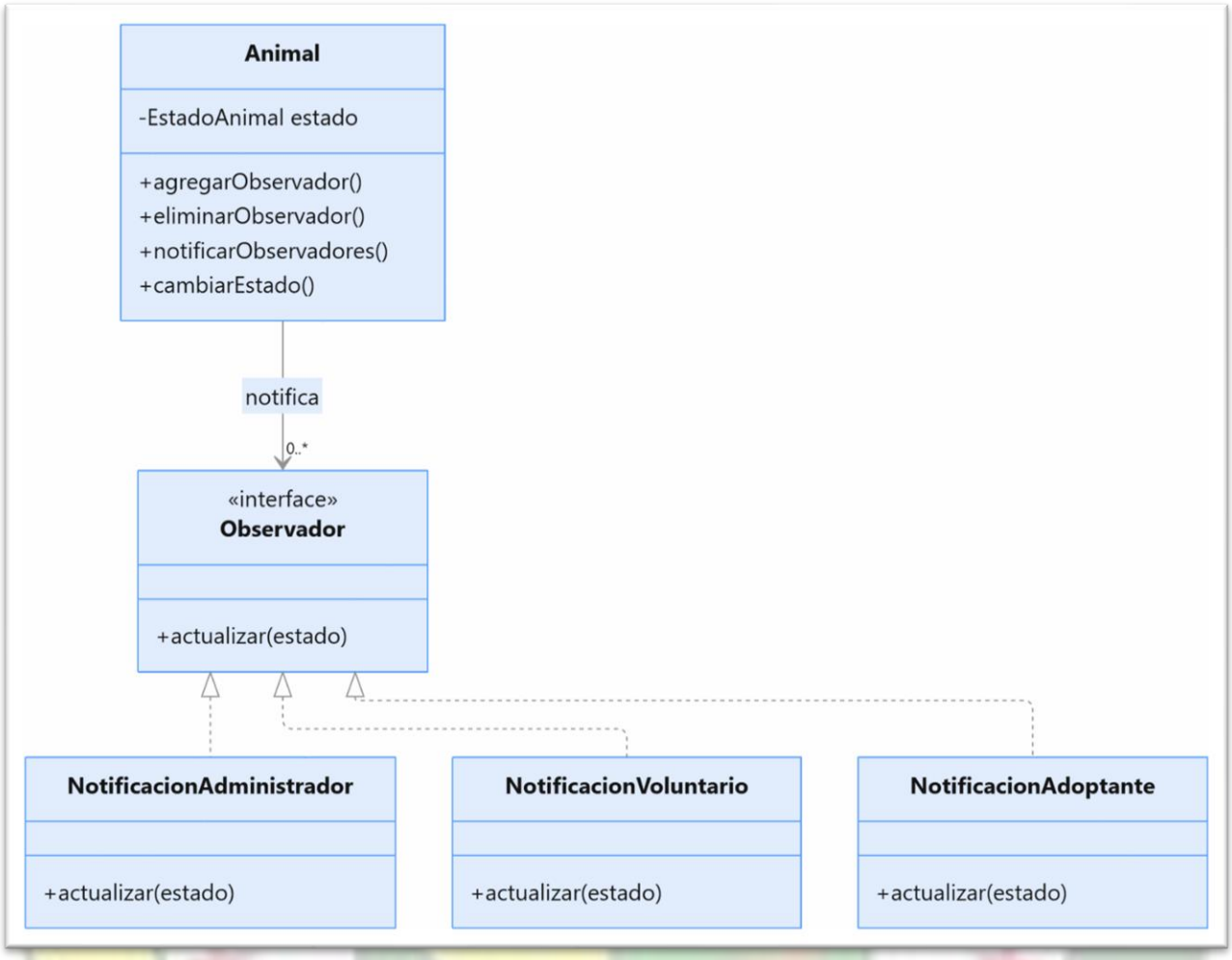
Ejemplo: Cuando se registra un nuevo rescate, la fachada coordina el registro del animal, almacena los datos del rescate, asigna un responsable y solicita una evaluación veterinaria.

Beneficio: Reduce la complejidad del sistema y evita que sus diferentes módulos dependan directamente unos de otros.

3.3. Patrón de comportamiento: Observer (Observador)

Propósito: Notificar automáticamente a los usuarios cuando se modifica el estado de un animal.

Se propone que la clase Animal actúe como sujeto observable, mientras que los usuarios interesados reciben notificaciones a través de una interfaz común.



Ejemplo: Cuando el veterinario determina que un animal terminó su tratamiento, su estado cambia a Disponible para adopción. El sistema notifica automáticamente a los usuarios interesados.

Beneficio: Permite mantener informados a los participantes sin que tengan que consultar constantemente el sistema.

4. Resumen de los patrones de diseño

Categoría	Patrón seleccionado	Aplicación
Creacional	Factory Method	Creación de animales según su especie.
Estructural	Facade	Coordinación de los procesos de rescate.
Comportamiento	Observer	Notificaciones automáticas por cambios de estado.

Consideración importante: Los animales silvestres deberán seguir un procedimiento de atención y derivación a las autoridades competentes, en lugar del proceso ordinario de adopción de mascotas.

5. Conclusión

El diseño del Sistema de Control de Rescate y Adopción Animal permite establecer una estructura organizada para administrar la información de los animales rescatados, sus tratamientos y los procesos de adopción.

Mediante el análisis de requerimientos se identificaron las necesidades principales del sistema y las responsabilidades de cada usuario. Asimismo, los diagramas UML facilitaron la representación de sus funcionalidades y clases.

Finalmente, la aplicación de los patrones Factory Method, Facade y Observer contribuye a desarrollar una arquitectura más flexible, reutilizable y fácil de mantener, permitiendo incorporar nuevas funcionalidades sin afectar considerablemente la estructura existente.

6. Referencias bibliográficas (APA 7)

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), https://www.omg.org/spec/UML/2.5.1

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Paso a paso Git y GitHub desde cero

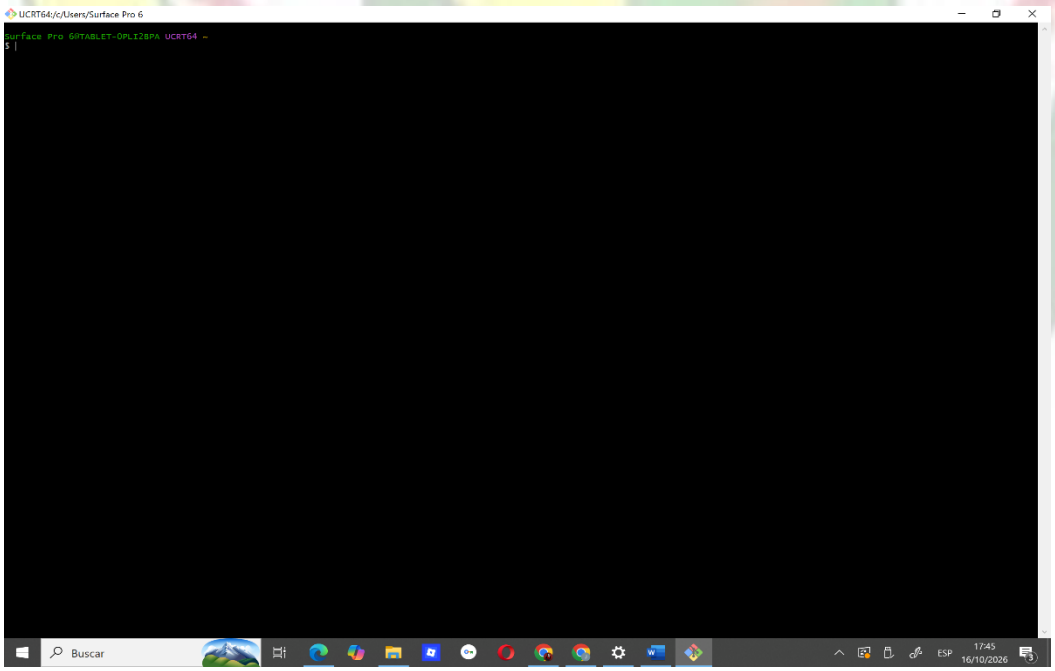
PASO 1. Abrir Git Bash

Como ya descargue Git:

1. Presiono el botón de inicio de Windows.

2. Escribo Git Bash.

3. Hago clic en la aplicación para abrirla.



Aparecerá una ventana donde escribiremos los comandos.

PASO 2. Configurar mi nombre de usuario y correo, recomendable nuestro correo institucional

Copiamos el siguiente comando, en Git Bash y presionamos Enter:

git config --global user.name "John"

Ahora configura tu correo**: git config --global user.email "jparrales5335@utm.edu.ec"**

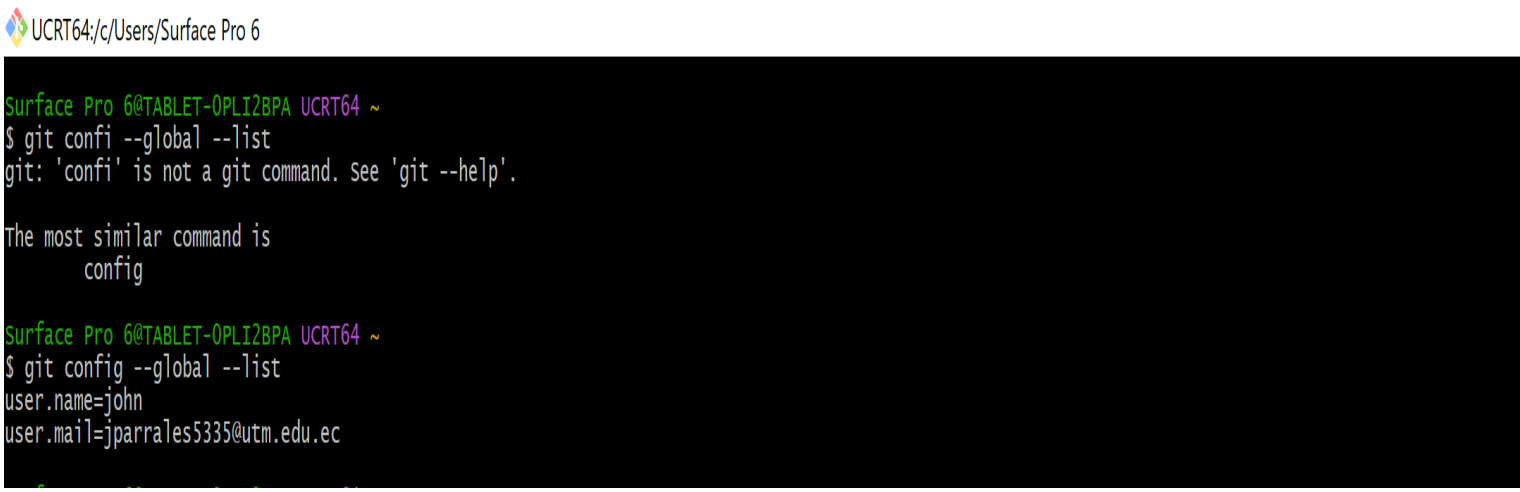
Para comprobar que se guardaron correctamente, escribe:

git config --global –list

Debería aparecer algo parecido a esto:

user.name=John

user.email=jparrales5335@utm.edu.ec



PASO 3. Abrir la carpeta de tu tarea

Ya tenemos una carpeta en el escritorio llamada:

desarrollo de sistemas informáticos

Dentro está guardo el documento:

john-parrales-estudio de caso

Hacemos lo siguiente:

1. Abrimos el Explorador de archivos de Windows.

2. Entramos al Escritorio.

3. Abrimos la carpeta desarrollo de sistemas informáticos.

4. Hacemos clic derecho.

5. Seleccionamos Open Git Bash here.

Esto abrirá Git directamente en la carpeta de mi trabajo.

Procedemos a crear un archivo de Git con el siguiente comando: **git init**

PARTE 3. Crear una cuenta en GitHub

Git y GitHub son diferentes. Git registra los cambios en tu computadora, mientras que GitHub permite almacenar tu proyecto en internet.

PASO 1. Registrarte en GitHub



GitHub

Página oficial: https://github.com

1. Ingresa a la página.

2. Haz clic en Sign up.

3. Introduce tu correo jparrales5335@utm.edu.ec.

4. Crea una contraseña segura.

5. Escoge un nombre de usuario que esté disponible. Por ejemplo, jparrales5335-hub.

6. Completa la verificación que solicite GitHub.

7. Confirma tu correo electrónico.

Importante: tu usuario de GitHub puede ser diferente del nombre configurado en Git Bash.

PASO 2. Crear un repositorio en GitHub

Cuando ingreses a tu cuenta:

1. Busca el símbolo + en la parte superior.

2. Selecciona New repository.

3. Completa la siguiente información:

Datos del repositorio

Nombre:

5smDesarrollo-de-Sistemas-Informaticos

Descripción:

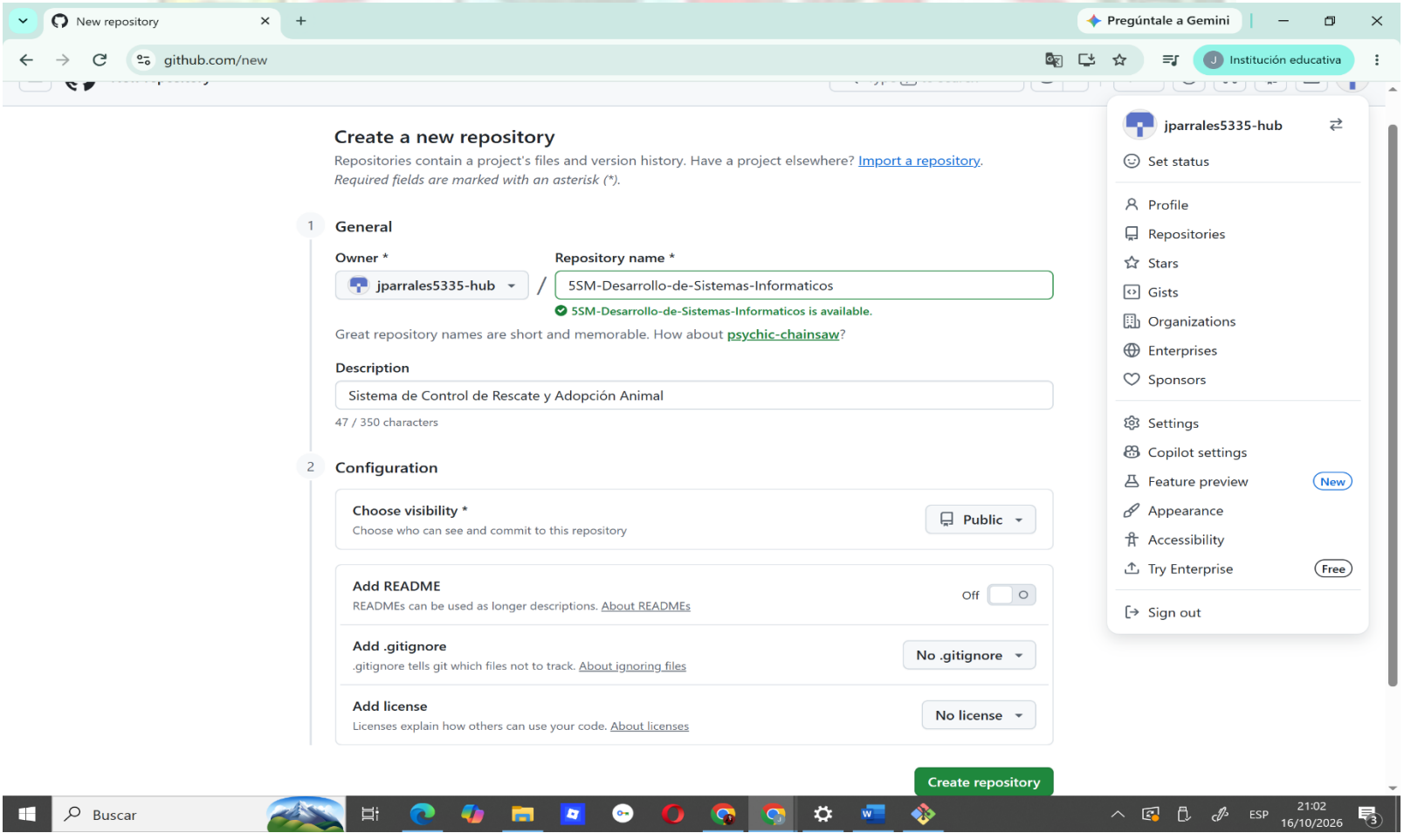
Sistema de Control de Rescate y Adopción Animal

Visibilidad:

Public

No marques las opciones de crear README, gitignore ni licencia.

Presiona Create repository.



PASO 3. Conectar Git Bash con GitHub

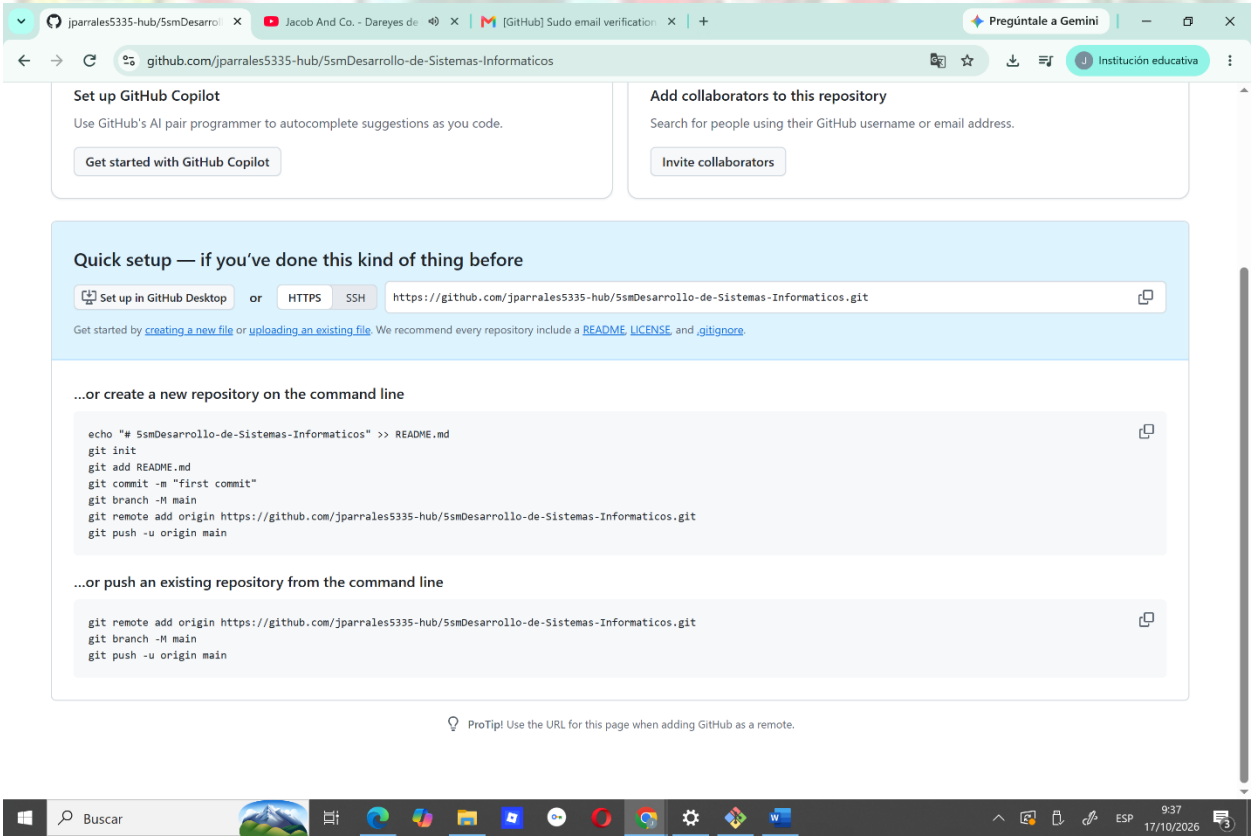
En la página de tu repositorio encontrarás una dirección HTTPS.

Cópiala.

Regresa a Git Bash y escribe: **git add . y aplastamos Enter**

Escribimos el siguiente comando**: git commit -m "commit inicial: primera version de documento"**

Y procedemos a copiar y pegar en Git Bash cada línea del comando que nos da el repositorio que ya hemos creado.



```
Surface Pro 6@TABLET-OPLI2BPA UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMA INFORMATICOS (main)
$ git add .

Surface Pro 6@TABLET-OPLI2BPA UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMA INFORMATICOS (main)
$ git commit -m "commit inicial: primera version de documento"
[main (root-commit) 1f7f7d0] commit inicial: primera version de documento
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 parrales-john-estudio-de-caso.lnk

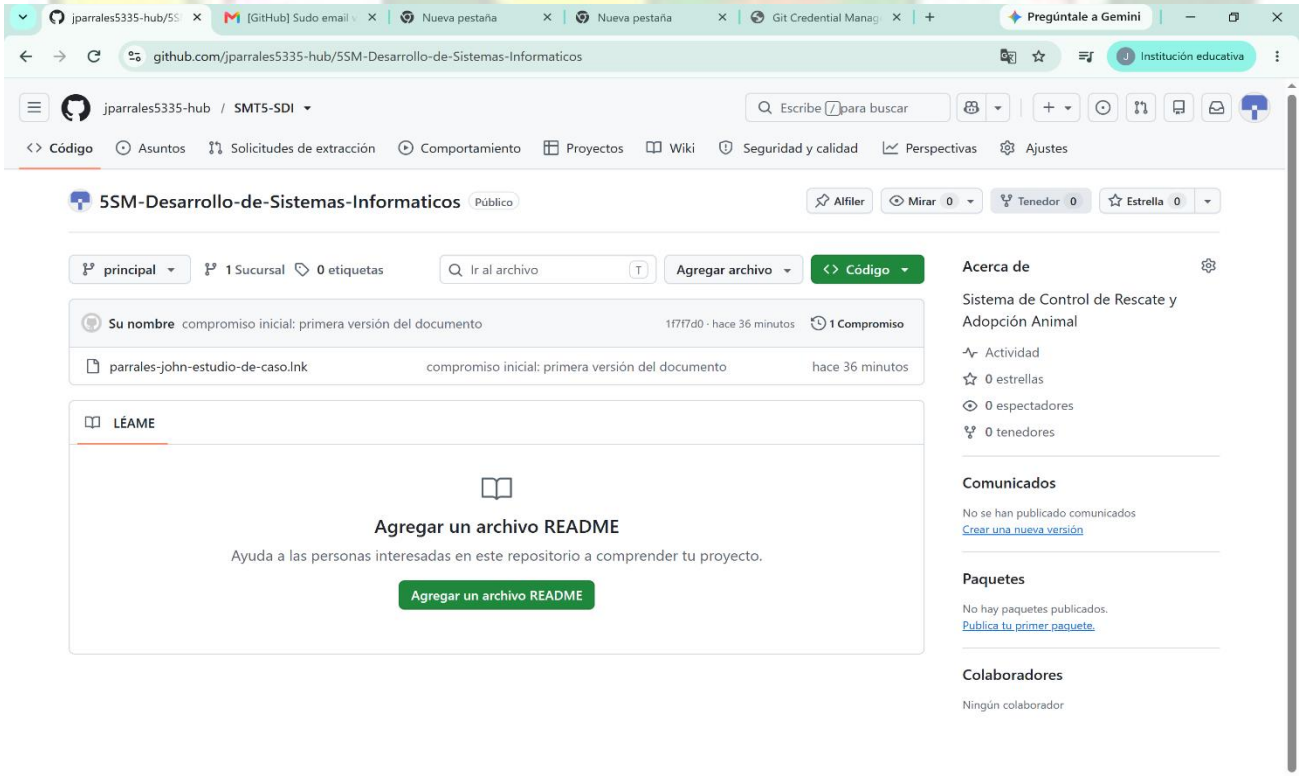
Surface Pro 6@TABLET-OPLI2BPA UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMA INFORMATICOS (main)
$ $ git commit -m "commit inicial: primera version de documento2"
bash: $: command not found

Surface Pro 6@TABLET-OPLI2BPA UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMA INFORMATICOS (main)
$ git remote add origin https://github.com/jparrales5335-hub/5SM-Desarrollo-de-Sistemas-Informaticos.git
error: remote origin already exists.

Surface Pro 6@TABLET-OPLI2BPA UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMA INFORMATICOS (main)
$ git branch -M main

Surface Pro 6@TABLET-OPLI2BPA UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMA INFORMATICOS (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 2.71 KiB | 2.71 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jparrales5335-hub/5SM-Desarrollo-de-Sistemas-Informaticos.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Y al actualizar nuestra rama podemos visualizar nuestro documento ya en GitHub.



Y cada que queramos actualizar entramos a Git Bash

Ponemos los siguientes comandos

1 **git add .** Este es un comando que avisa al GitHub que se va a realizar un cambio.

2 **git commi -m "segundo cambio: se elimino el cuarto párrafo"** : Este el comando de aviso de que se va hacer el cambio por ejemplo se va a eliminar algo o agregar algo al archivo conjunto son un comentario.

3 y por ultimo el comando **git push** que confirma a GitHub que ya se efectuó un cambio.



Muchas gracias